S05 Audio

CRE132 worksheet

What you'll make

Sound for S03's room: looping music, a pickup sound for coins, a hurt sound in the fire and a click for the Restart button. Two sliders in the bottom left set the music and effects volume separately.

By the end you'll know the four parts of Unity audio (**Audio Listener**, **Audio Source**, **Audio Clip** and **Audio Mixer**), why sounds go through an **AudioManager** singleton, and how a slider talks to a mixer.

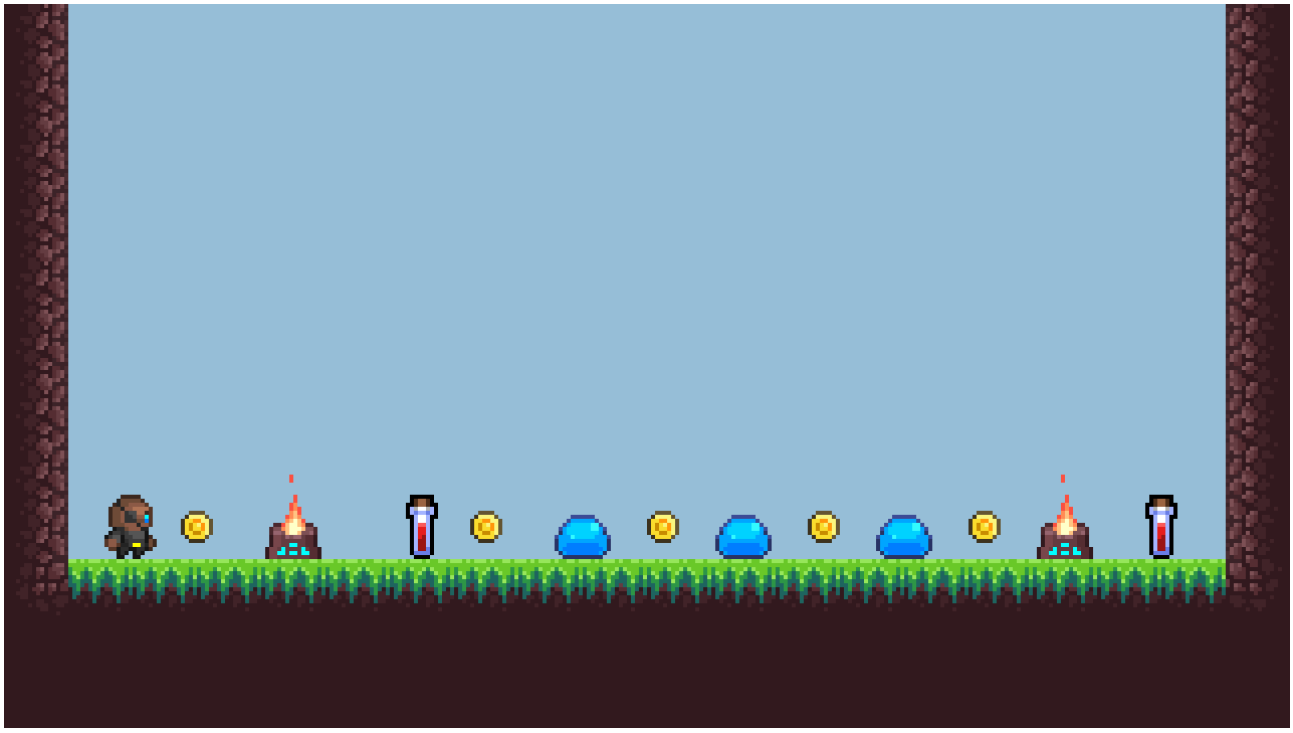


Figure 1: The finished scene: the player, coins, two fires, two potions and three training dummies.

Before you start

1. Unzip `CRE132_S05_Audio_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_S05_Audio_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Shared/S05_Audio`.
3. Press **Play**. Everything from S03 works (health bar, fires, potions, J to attack, Restart), except the coins, which do nothing yet. It's silent, and the **Music** and **Effects** sliders in the bottom left do nothing. Press Play again to stop.

The scene also has an **AudioManager** (from `Assets/_game/Prefabs/Managers/AudioManager.prefab`) with two children, **Music Source** and **Effects Source**. The sounds are in `Assets/_game/Audio/Music` and `Assets/_game/Audio/SFX`, and the mixer is `Assets/_game/Audio/Mixers/MainMixer`.

> **Making a script:** in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. Ears, speakers and the sound desk

Have a look at how the audio is set up before writing any code.

1. **Audio Listener (the ears):** select **Main Camera**. It has an **Audio Listener**. A scene needs exactly one; with none, you hear nothing and the Console warns you.
2. **Audio Source (a speaker):** select **Music Source** inside the AudioManager. **Loop** is ticked, and its **Output** is the mixer's **Music** group. **Effects Source** has **Loop** off and Output **SFX**. To hear an Audio Source with no code at all, drop a clip in its **Audio Resource** slot and tick **Play On Awake**.
3. **Audio Clip (the sound file):** select `Placeholder_Chiptune_Loop` in `Audio/Music`. Press the play button in the Inspector's preview. **Load Type** decides how it's stored: short effects are fine as **Decompress On Load**; for long music, **Streaming** saves memory.
4. **Audio Mixer (the sound desk):** double-click `MainMixer`. Under **Master** are two groups, **Music** and **SFX**, each with its own fader. At the top right, **Exposed Parameters (2)** lists `MusicVolume` and `SFXVolume`: those are the faders a script is allowed to move. (To expose one yourself: select a group, right-click **Volume** in the Inspector and choose **Expose ... to script**, then rename it in that list.)

2. The AudioManager

Create a script `AudioManager` in `Assets/_game/Scripts/Managers`:

```
// The AudioManager plays the music and the sound effects, and sets their volumes
// through an Audio Mixer.
// Like the GameManager there is one of it, it survives scene loads, and any script
// can reach it with AudioManager.Instance.
using UnityEngine;
using UnityEngine.Audio;

public class AudioManager : MonoBehaviour
{
    public static AudioManager Instance { get; private set; }

    [SerializeField] private AudioMixer mixer;

    // Two AudioSources: one loops the music, the other plays short effects. Their
    // Output is set to the Music and SFX groups of the mixer, so each has its own
    // volume.
    [SerializeField] private AudioSource musicSource;
    [SerializeField] private AudioSource effectsSource;

    [SerializeField] private AudioClip startingMusic;

    private void Awake()
    {
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }

        Instance = this;
        DontDestroyOnLoad(gameObject);
    }

    private void Start()
    {
        if (startingMusic != null)
```

```
        {
            PlayMusic(startingMusic);
        }
    }

    public void PlayMusic(AudioClip music)
    {
        // Already playing this track (for example after a scene reload): let it carry
        // on instead of restarting.
        if (musicSource.clip == music && musicSource.isPlaying)
        {
            return;
        }
        musicSource.clip = music;
        musicSource.Play();
    }

    // PlayOneShot lets sounds overlap: collecting two coins quickly plays two coin
    // sounds.
    public void PlaySound(AudioClip sound)
    {
        effectsSource.PlayOneShot(sound);
    }

    // Volume from a slider (0 to 1) to the mixer, which works in decibels (-80 to 0).
    public void SetVolume(string exposedParameter, float sliderValue)
    {
        // Log10(0) can't be worked out, so keep the slider value just above zero.
        if (sliderValue < 0.0001f)
        {
            sliderValue = 0.0001f;
        }
        float decibels = Mathf.Log10(sliderValue) * 20f;
        mixer.SetFloat(exposedParameter, decibels);
    }
}
```

What's going on:

- **A singleton** like S02's GameManager, so any script can write `AudioManager.Instance.PlaySound(...)`, and the music keeps going when the scene reloads.
- **`PlayMusic`** checks whether that track is already playing and leaves it alone, so Restart doesn't start the music again from the beginning.
- **`PlayOneShot`** lets sounds overlap. Setting `.clip` and calling `Play()` instead would cut the last sound off.
- **`SetVolume`** turns a slider value (0 to 1) into decibels with `Mathf.Log10(value) * 20`: 1 is 0 dB (full), 0.5 is about −6 dB, which sounds about half as loud. `mixer.SetFloat` moves the exposed fader.

Add it: open the **AudioManager** prefab (double-click it in `Prefabs/Managers`), select the root **AudioManager** and add **Audio Manager**. Fill the slots: **Mixer** `MainMixer`, **Music Source** the **Music Source** child, **Effects Source** the **Effects Source** child, **Starting Music** `Placeholder_Chiptune_Loop`. Save the prefab and go back to the scene.

Test it: press Play. The music starts and loops. Open the Mixer window during Play and drag the Music fader, or click **S** (solo) and **M** (mute) on a group.

3. A coin with a sound: SoundCoin

Make a folder `Assets/_game/Scripts/S05_Audio` and create `SoundCoin` in it:

```
// S02's ScoreCoin with a sound: adds to the score and plays a pickup sound through
// the AudioManager.
using UnityEngine;

public class SoundCoin : MonoBehaviour
{
    [SerializeField] private int value = 1;
    [SerializeField] private AudioClip pickupSound;

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("Player"))
        {
            GameManager.Instance.AddScore(value);

            // The coin is about to be destroyed, so it can't play the sound itself
            // (its AudioSource would go with it).
            AudioManager.Instance.PlaySound(pickupSound);
            Destroy(gameObject);
        }
    }
}
```

- **Why the coin doesn't have its own Audio Source:** it's destroyed straight away, and its Audio Source would be destroyed with it, cutting the sound off. The AudioManager outlives the coin.

Add it to all five **Coin** objects inside **Coins**. Leave **Value** at 1 and set **Pickup Sound** to `GS2_Item_Acquire_1` (from `Audio/SFX`).

4. Ouch: HurtSound

Create `HurtSound` in the `S05_Audio` folder:

```
// Plays a sound whenever this object's Health goes down.
// It watches the number each frame; S09 shows how Health could announce it instead.
using UnityEngine;

public class HurtSound : MonoBehaviour
{
    [SerializeField] private AudioClip hurtSound;

    private Health health;
    private int lastHealth;

    private void Start()
    {
        health = GetComponent<Health>();
        lastHealth = health.CurrentHealth;
    }

    private void Update()
    {
        if (health.CurrentHealth < lastHealth)
        {
            AudioManager.Instance.PlaySound(hurtSound);
        }
        lastHealth = health.CurrentHealth;
    }
}
```

- **`lastHealth`** remembers last frame's health. If it's lower now, something hurt us, so play the sound. Healing makes it higher, so that's silent.

Add it to the **Player** and set **Hurt Sound** to `GS1_Damage_1`.

5. A click: ButtonSound

Create `ButtonSound` in the `S05_Audio` folder:

```
// Put on a UI Button to play a click sound when it's pressed.
using UnityEngine;
using UnityEngine.UI;

public class ButtonSound : MonoBehaviour
{
    [SerializeField] private AudioClip clickSound;

    private void Start()
    {
        Button button = GetComponent<Button>();
        button.onClick.AddListener(PlayClick);
    }

    private void PlayClick()
    {
        AudioManager.Instance.PlaySound(clickSound);
    }
}
```

- **`onClick.AddListener`** adds `PlayClick` to the button's **On Click ()** list from code, so you don't have to set it up in the Inspector. The Restart entry already in the list still runs too.

Add it to **Restart Button** and set **Click Sound** to `SciFi_UI_Button_1`.

Test it: press Play. Coins ping, the fire says ouch once a second, and Restart clicks. After a restart the music carries on without starting again.

6. Volume sliders: VolumeSlider

Create `VolumeSlider` in `Assets/_game/Scripts/UI`:

```
// Put on a UI Slider (0 to 1) to control one volume in the Audio Mixer, such as
// "MusicVolume".
using UnityEngine;
using UnityEngine.UI;

public class VolumeSlider : MonoBehaviour
{
    // The name the volume was exposed with in the Audio Mixer (right-click Volume >
    // Expose).
    [SerializeField] private string exposedParameter = "MusicVolume";

    private Slider slider;

    private void Start()
    {
        slider = GetComponent<Slider>();

        // Call SetVolume whenever the slider moves. This is the same list as On Value
        // Changed in the Inspector.
```

```
        slider.onValueChanged.AddListener(SetVolume);
        SetVolume(slider.value);
    }

    private void SetVolume(float value)
    {
        AudioManager.Instance.SetVolume(exposedParameter, value);
    }
}
```

- **`onValueChanged.AddListener(SetVolume)`** calls `SetVolume` with the new value every time the slider moves.
- **`exposedParameter`** is the name from the mixer's Exposed Parameters list. It's a string, so it must match exactly, capitals included.
- **The sliders' range is 0 to 1** (**Min Value** 0, **Max Value** 1, **Whole Numbers** off), to match `SetVolume`.

Add it: in **Volume Panel**, add **Volume Slider** to **Music Slider** and leave **Exposed Parameter** as `MusicVolume`. Add it to **Effects Slider** and change it to `SFXVolume`.

Test it: press Play. The Music slider turns only the music down; the Effects slider only the coin, hurt and click sounds.

Check your work

- ☐ Music starts when you press Play, loops, and doesn't restart when you click Restart.
- ☐ Coins, the fire and the Restart button each make their sound, and quick coins overlap.
- ☐ Each slider changes only its own group.
- ☐ Compare with `CRE132_S05_Audio_Finished` if something behaves differently.

Key points

- **Listener** (ears), **Source** (speaker), **Clip** (the sound file), **Mixer** (the sound desk).
- **Sounds that must outlive their object** (pickups, deaths) go through a manager.
- **Mixer groups** let players set music and effects volume separately, which every settings menu needs. Scripts can only move a mixer value once it's been exposed.
- **The sliders start at full every time the scene loads**, so Restart sets the volume back to full. S06 and S08 fix that.

If something goes wrong

- **No sound at all:** check the Game view's **Mute Audio** button (the speaker icon in its toolbar), and that Main Camera still has its Audio Listener.
- **NullReferenceException in SoundCoin, HurtSound, ButtonSound or VolumeSlider:** `AudioManager.Instance` is `null`. Check the AudioManager component is on the prefab, not just added and then lost when you left prefab mode without saving.
- **NullReferenceException in AudioManager:** one of its slots is empty.
- **A slider does nothing:** the **Exposed Parameter** name doesn't match the mixer's (`MusicVolume`, `SFXVolume`), or the Audio Source's **Output** isn't set to a mixer group.
- **"PlayOneShot was called with a null AudioClip":** a sound slot (Pickup, Hurt or Click Sound) is empty.

Try it

1. Add a jump sound: an `AudioClip jumpSound` field and a `PlaySound` call in a copy of `InputSystemJump`.
2. Randomise the pitch a little so repeated coins sound less robotic: in `PlaySound`, set `effectsSource.pitch = Random.Range(0.9f, 1.1f);` before `PlayOneShot`.

3. Add an **Ambience** group to the mixer with a looping background sound, expose its volume, and add a third slider.